Literature Review Outline Template

Project Idea:

PentraAI — AI-Driven Autonomous Web Penetration Testing

Group Members:
Prabanjan Velayutham
Pooja Sri Kuppuswamy Niranjana
Muhammad Saad Abdullah
Mohammed Iliyaz

1. Problem Context

- **What problem are you addressing?**
  Current web security relies on tools like Burp Suite, OWASP ZAP, and Nuclei that excel at detecting known vulnerability signatures like SQL injection, cross-site scripting, and basic misconfigurations. However, they are fundamentally incapable of detecting vulnerability classes that require contextual reasoning: Insecure Direct Object References (IDOR/BOLA), Broken Authentication logic (JWT algorithm confusion, OAuth misconfigurations), and Race Conditions. These categories consistently top the OWASP Top 10:2025 and are responsible for the most damaging real-world breaches, yet no mainstream tool automates their detection without expert human configuration at every step.
- **Why is it important?**
  As web applications grow in complexity, manual pentesting cannot scale to match the pace of development. OWASP API Security Top 10 data shows that IDOR/BOLA (A01) alone accounts for the majority of API security incidents. Automating reasoning-based vulnerability detection using AI means organisations can find these critical flaws faster, more consistently, and without requiring a senior security engineer for every scan. PentraAI directly addresses this capability gap by replacing rule-matching with LLM-driven contextual reasoning across the full pentest pipeline.

2. Overview of Existing Work

- **What solutions already exist?**
  We primarily use DAST (Dynamic Application Security Testing) tools like OWASP ZAP and Nuclei for vulnerability scanning, and Burp Suite Professional for manual exploitation.
- **What methods/models/systems are commonly used?** Most current tools use "signature-based" matching looking for specific code patterns. Recently, there's been a shift toward using Reinforcement Learning (RL) for autonomous testing and early LLM-based assistants like PentestGPT to help human testers.

3. Comparison of Approaches

- **What are the strengths and weaknesses of these approaches?**
  Traditional tools (ZAP, Nuclei) are fast and have low false positives for simple bugs like XSS, but they are "blind" to application logic. They can't understand that "User A" shouldn't be able to see "User B's" receipt. RL approaches are better at navigation but often struggle with the vast state space of modern web apps.
- **How do they differ?**
  The fundamental distinction is between detection and reasoning. Existing tools detect symptoms a response matches a pattern. PentraAI reasons about intent: it reads the application, builds a model of what it is supposed to do, hypothesises what could be abused, executes targeted tests, and interprets whether the response demonstrates actual exploitation. This mirrors how a skilled human pentester approaches a target, rather than how a scanner does.

4. Identified Gap

- **What is missing, limited, or not working well?**
  There is no fully automated pipeline that can crawl an app, build a mental model of how it works, hypothesize a complex attack, and then execute it without a human clicking "send".
- **Where is the opportunity for improvement?**
  Large Language Models (LLMs) provide the "brain" that was missing. By using an agentic framework like Lang Graph, we can move past simple pattern matching and start automating actual security "thinking".

5. Positioning Your Project

- **How does your project address the gap?**
  PentraAI uses an LLM-driven "Agentic Pipeline". Instead of just scanning, it performs Recon, forms a Hypothesis, executes a targeted Test, and then analyzes the response just like a human would.
- **What approach will you take?**
  PentraAI is built on a Python/FastAPI backend with a LangGraph agent managing the five-phase pipeline. The LLM layer uses Llama 3.3-70b-versatile via the Groq API (free tier). The tool specifically targets three OWASP Top 10:2025 categories where the automation and reasoning gap is largest: IDOR/BOLA (A01), Broken Authentication including JWT algorithm confusion and OAuth misconfigurations (A07), and Race Conditions (A06). Recon employs five discovery techniques in parallel: common path probing, Wayback Machine historical URL lookup, wordlist fuzzing with 200+ API paths, subdomain enumeration via DNS resolution, and deep JavaScript analysis to extract embedded API calls from bundled frontend code.

6. Research to Application Mapping

- **Insight 1:** LLMs hallucinate in security contexts without grounding
  Application: We are implementing a "Response Analysis" phase where the LLM must provide evidence and reasoning to verify a finding before reporting it.
- **Insight 2:** Effective race condition testing requires precise "last-byte" synchronization.
  Application: We will use httpx to send concurrent requests while using the LLM to

identify which specific endpoints are most likely to be vulnerable to timing attacks.

- **Insight 3:** Multi-step reasoning needs "state" management.
  Application: We're using Lang Graph to allow the agent to loop back and re-test if a hypothesis isn't initially proven.

7. Key References

- OWASP Top 10:2025: authoritative source for target vulnerability categories and prevalence data
- OWASP API Security Top 10:2023: detailed attack patterns for IDOR/BOLA (API1), Broken Authentication (API2), and security misconfiguration
- Deng et al. (2023). PentestGPT: Evaluating LLMs for Automated Penetration Testing primary comparison system; demonstrates LLM advisory capability and its manual execution limitation
- Happe & Cito (2023). Getting pwn'd by AI: Penetration Testing with Large Language Models empirical analysis of LLM capability boundaries in security tasks
- PortSwigger Research (2023). Race Conditions: New Classes and the Single-Packet Attack (Black Hat USA 2023) technical foundation for PentraAI's race condition timing implementation
- Humeau et al. (2023). LangGraph: Building Stateful Multi-Actor Applications architectural basis for PentraAI's agentic pipeline implementation
- Celière et al. (2022). JWT Security Taxonomy and Common Attack Vectors: reference for Broken Authentication module design (alg:none, weak secret, privilege escalation)
- Verizon (2024). Data Breach Investigations Report: provides breach prevalence data supporting the importance of IDOR and authentication flaws
- Groq Inc. (2025). Llama 3.3-70b-versatile API Documentation: technical reference for the LLM inference layer used in all three reasoning phases

8. Conclusion

- **What did you learn from the literature?**
  The literature shows that while AI is being used as an *assistant*, the jump to a fully *autonomous* agent is still very new and technically difficult especially regarding logic flaws.
- **Did the learning influence your direction? Briefly explain how?**
  The research confirmed the decision to focus exclusively on the three OWASP categories with the largest automation gap IDOR (A01), Broken Authentication (A07), and Race Conditions (A06) rather than building another scanner for SQLi or XSS where ZAP and Nuclei already perform well. It also validated the architectural decision to use LLM reasoning only at decision points (hypothesis, analysis, reporting) rather than for all tasks, keeping costs near zero while maximising the reasoning benefit. The PortSwigger race condition research directly shaped the implementation of the timing attack module. Most importantly, the literature confirmed that the five-phase autonomous pipeline PentraAI implements Recon, Hypothesis, Test, Analysis, Report represents a genuinely novel contribution rather than an incremental improvement on existing tools.